

SUPPLEMENTARY PROBLEM: MATRIX CHAIN MULTIPLICATION

Instructor: Orhan Özgüner

© Özgüner @CWRU, 2025

Problem Statement:

Given a sequence of matrices $A_1, A_2, \dots, A_n$, where matrix A_i has dimensions $p_{i-1} \times p_i$, fully parenthesize the product $A_1 A_2 \cdots A_n$ in a way that minimizes the number of scalar multiplications. By traditional matrix multiplication, the cost of multiplying two matrices A and B with dimensions $p \times q$ and $q \times r$, respectively, is pqr .

0.1 Example**Input:**

A sequence of matrices with dimensions:

- $A_1 : 10 \times 30$
- $A_2 : 30 \times 5$
- $A_3 : 5 \times 60$

Output:

- $(A_1 A_2) A_3$
- Minimum number of scalar multiplications: 4500

Problem Solution

Recurrence Equation

$$M_{i,j} = \begin{cases} 0 & \text{if } i = j, \\ \min_{i \leq k < j} \{M_{i,k} + M_{k+1,j} + p_{i-1}p_kp_j\} & \text{if } i < j \end{cases}$$

Proof of Optimal Substructure

If we are trying to solve for the minimum number of scalar multiplications given arrays $A_1..A_n$, then the subproblem in this context is the minimum number of scalar multiplications for some contiguous subarray of matrices $A_i..A_j$ where $1 \leq i \leq j \leq n$. Also, the subproblems are independent of each other since the set of subarrays being considered at each level is split into two groups, i.e. a matrix can only be apart of one product at a time.

Given: Let S be an optimal solution for an instance of the matrix chain problem with input matrices $A_{i..j}$.

Claim: There must be two terms in the product of the outermost parenthesization, i.e. $S = A_{i..j} = A_{i..k}A_{k+1..j}$ for $i \leq k < j$. Then, claim that $A_{i..k}$ is an optimal solution for the matrix chain problem (you could pick $A_{k+1..j}$ and prove by the same argument).

Proof by Exchange of Arguments: Assume that our claim is **false**. This means that $A_{i..k}$ is **not** optimal solution. Then, there must be an alternative optimal solution $A'_{i..k}$ that is feasible and provides a lower number of scalar multiplications than $A_{i..k}$. Now, reconstruct the optimal solution for the original problem in the following way:

$$S^* = A'_{i..k}A_{k+1..j}$$

Here, S^* is a feasible solution for the original matrix chain multiplication problem, and it provides a lower total number of scalar multiplications than the given optimal solution S . This creates a contradiction, as S was assumed to be optimal.

Therefore, $A'_{i..k}$ cannot be constructed, and S' must be an optimal solution for the matrix chain multiplication problem with arrays $A_{i..j}$.

Pseudocode

Algorithm 1 Matrix-Chain-Order(p, n)

```
1:  $s, m \leftarrow$  matrix of dimensions  $n \times n$ 
2: for  $i \leftarrow 1$  to  $n$  do ▷ Base Cases
3:    $m[i, i] \leftarrow 0$ 
4: end for
5: for  $l \leftarrow 2$  to  $n$  do ▷  $l$  is the chain length
6:   for  $i \leftarrow 1$  to  $n - l + 1$  do
7:      $j \leftarrow i + l - 1$ 
8:      $m[i, j] \leftarrow \infty$ 
9:     for  $k \leftarrow i$  to  $j - 1$  do
10:       $q \leftarrow m[i, k] + m[k + 1, j] + p_{i-1}p_kp_j$ 
11:      if  $q < m[i, j]$  then
12:         $m[i, j] \leftarrow q$ 
13:         $s[i, j] \leftarrow k$ 
14:      end if
15:    end for
16:  end for
17: end for
18: return  $m$  and  $s$ 
```

Runtime Analysis

We need to consider

1. The number of subproblems
2. The amount of work done at each subproblem

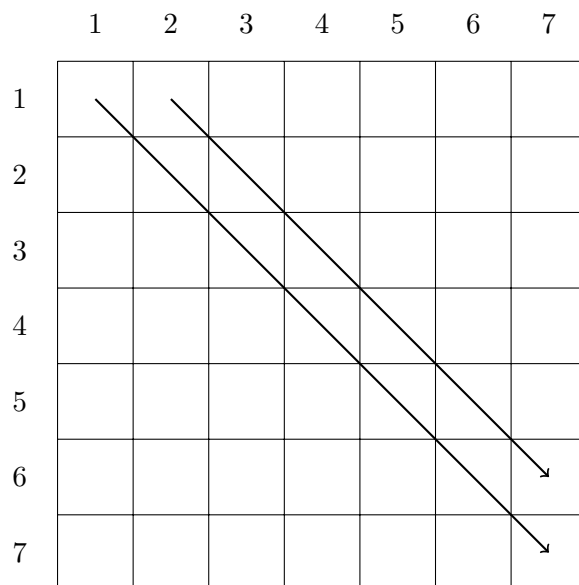
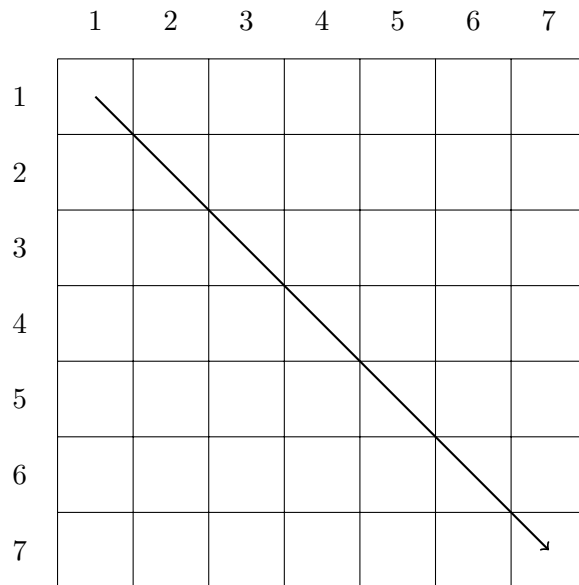
Since we are storing the subproblems in a matrix, that leads us to believe that we are calculating n^2 subproblems. Before making this assumptions, let's understand further the order in which we are solving subproblems. We know first that the base cases are being loaded into the matrix for entries that are on the diagonal. This is depicted in the diagram above.

From there, we start to calculate subproblems for subarrays of size 2, i.e. when $i = j + 1$. This can be reflected in the diagram below.

We will keep loading values in this diagonal manner until the top right half of the matrix is completely full. Therefore, mathematically, the number of subproblems can be explained as so

$$\# \text{ of subproblems} = n + n - 1 + \dots + 1 = \frac{n(n+1)}{2}$$

At each subproblem $M[i, j]$, the pseudocode takes into consideration all $j - i$ parenthesizations. In the worst case, $j - i$ will equal $n - 1$. If we correct memoize the solutions to subproblems, then we can access them in constant time. Therefore, if we are doing an order of n work at each subproblem, and there are an order of n^2 subproblems, the runtime is



$$\Theta(n^3)$$